Lecture 05

RISC-V: Procedures

Instructor: Dr. Muhammad Imran

CHIPXPRT

NCDC

NUST CHIP DESIGN CENTRE

CX-204: Modern Computer Architecture

Acknowledgement

- Content from following has been used in these lectures
  - Computer Organization and Design, RISC-V 2nd Edition, Patterson and Hennessy
  - CS 61C: Great Ideas in Computer Architecture (Machine Structures), UC Berkeley

Contents

- Functions in C
- Stack
- RISC-V Functions
- Calling Convention

C Functions

```
int foo(int i) {
    if(i == 0) return 0;
    int a = i + foo(i-1);
    return a;
}
int j = foo(3);
int k = foo(100);
int m = j+k;
```

Two jumps for each function: Jump to the function for the function call, and jump back to the next line of code after the function returns

C Functions

```c
int foo(int i) {
    if(i == 0) return 0;
    int a = i + foo(i-1);
    return a;
}
int j = foo(3);
int k = foo(100);
int m = j+k;
```

- Calling a function:
  - Set function arguments
  - Goto the start of the function
- During a function call:
  - Keep local scope separate from global scope
  - Perform the desired task of the function
- Returning from a function:
  - Place the return value in a variable that can be accessed
  - Goto the line immediately after the function call

Problem with Maintaining Scope

- In RISC-V, local scope doesn't exist; all registers are "kept" throughout the program
  - If a function changes register x10, then the global value of x10 will also change
- Can we solve this by just making sure each function uses a different set of registers?
  - No; recursive function calls won't be able to use different registers
- We'll need a way to store variables somewhere that no called function can change

Problem with returning from a function

- In C, all **gotos** need to go to a specific label (that can't change)
  - However, when returning from a function, we need to jump to different places depending on who called the function (the return address)
  - This can be solved if we treat the return address as an input to the function

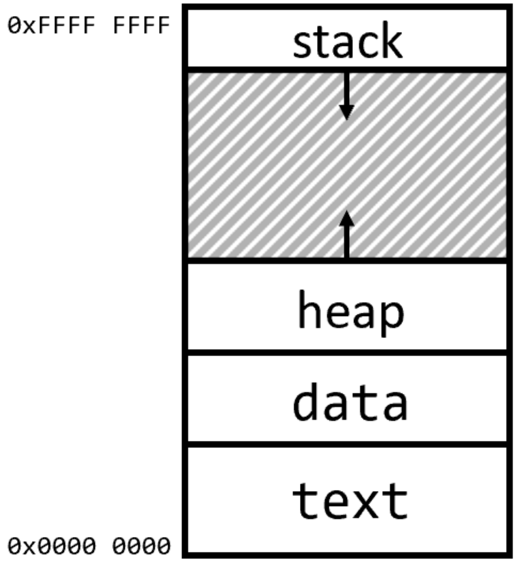
- C doesn't actually let you store a label in a variable/argument, so we won't be able to reduce functions in C using just **gotos**

- We'll need a way to send in the return address to a function and jump to that return address when we finish with the function.

Review: C Memory Model

- In C, memory was divided into four segments:
  - Code/Text
  - Static/Data
  - Heap
  - Stack

- RISC-V uses the same memory layout. Today, we'll take a c d stack segments!



Text

- RISC-V code is also a form of data. This data gets stored in the text section of memory
- In RISC-V, every (real) instruction is stored as a 32-bit number
- Thus, the "next" instruction is always stored 4 bytes after the current instruction.
- A special 33rd register called the Program Counter (or PC) keeps track of which line of code is currently being run.

Current Line

Address	Data
0x0000 0000	addi x5 x0 5
0x0000 0004	xor x5 x6 x6
0x0000 0008	jal x1 Label
0x0000 000C	sw x5 8(x2)
0x0000 0010	beq x0 x0 XX
0x0000 0014	bne x0 x0 XX

Register	Value
PC	0x0000 0008

RISC-V **Jump Instructions**

- The address of an instruction can be used (along with the PC) to perform the jumps we need for functions
  - jal rd Label
  - Jump And Link
    - Jumps to the given label, but also sets rd to PC+4 (the line after the current line)

  - Ex. If we run the current line, **x1** will be set to **0x0000 000C**, and PC will move to Label.

Current Line

Address	Data
0x0000 0000	addi x5 x0 5
0x0000 0004	xor x5 x6 x6
0x0000 0008	jal x1 Label
0x0000 000C	sw x5 8(x2)
0x0000 0010	beq x0 x0 XX
0x0000 0014	bne x0 x0 XX

Register	Value
PC	0x0000 0008

RISC-V Jump Instructions

- **jal rd Label** Jump And Link
  - Jumps to the given label, but also sets **rd** to **PC+4**
  - (the return address)
  - Often used for function calls

- **j Label** Jump
  - (From last lecture) Jumps to the given label. Pseudoinstruction for **jal x0 Label**
  - Used for unconditional jumps (ex. loops)

RISC-V Jump Instructions

- **jalr rd rs1 imm** Jump and Link Register
  - Jumps to the instruction at address **rs1+imm**, and sets **rd** to **PC+4**
  - Less common than other jumps, but used for higher-order functions and some function calls

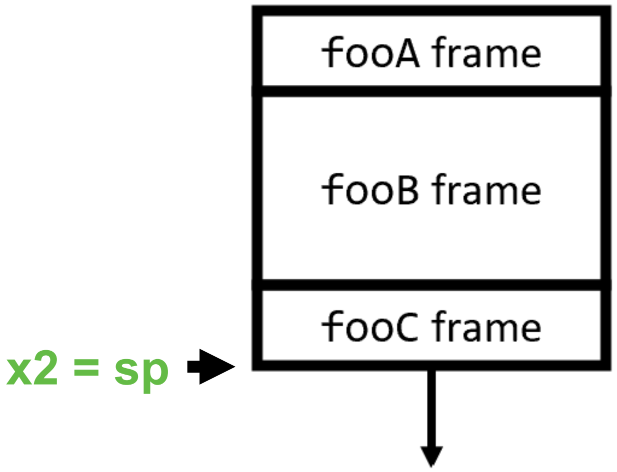
- **jr rs1** Jump to Register
  - Jumps to the instruction at address rs1
  - Also, a pseudoinstruction for **jalr x0 rs1 0**
  - Often used to return from a function

Stack

- In C: Each function call automatically creates a stack frame, with nested calls growing the stack downward.

- In RISC-V: One of our registers (by convention **x2**, nicknamed **sp**, or "stack pointer") is set to the bottom of the stack. A function can choose to create a stack frame, by manipulating **sp**!

```
fooA() { fooB(); }
fooB() { fooC(); }
fooC() { … }
```



RISC-V: Rules for Manipulating the Stack

- Anything above the **sp** at the start of a function belongs to another function. You may not modify anything above the sp without permission.
- Everything below the **sp** is safe to modify.
- But anyone else can modify it, so you can't leave data there and expect it to stay the same
- By decrementing the **sp**, we can allocate as much space as we need for our function, that we can use however we want.
- After finishing a function call, the **sp** must be set to its value from before the function call

Address	Data
0xFFFF FF0C	
0xFFFF FF08	
0xFFFF FF04	
0xFFFF FF00	
0xFFFF FEFC	
0xFFFF FEF8	

Register	Value
sp	0xFFFF FF04

Manual Stack Manipulation: Example

```
fooB:
addi sp sp -8
...
jal x1 fooC
...
addi sp sp 8
jr ra
```

Current Line

Space that we aren't allowed to change

By convention, stack addresses written from greatest to smallest

Address	Data
0xFFFF FF0C	0x12345678
0xFFFF FF08	0x9ABCDEF0
0xFFFF FF04	0x00000000
0xFFFF FF00	0xABADCAFE
0xFFFF FEFC	0x4F639DAB
0xFFFF FEF8	0x14857642

Register	Value
sp	0xFFFF FF04

Manual Stack Manipulation: Example

```
fooB:
addi sp sp -8
...
jal x1 fooC
...
addi sp sp 8
jr ra
```

Current Line

Space that we can change however we want

Address	Data
0xFFFF FF0C	0x12345678
0xFFFF FF08	0x9ABCDEF0
0xFFFF FF04	0x00000000
0xFFFF FF00	0xABADCAFE
0xFFFF FEFC	0x4F639DAB
0xFFFF FEF8	0x14857642

Register	Value
sp	0xFFFF FEFC

Manual Stack Manipulation: Example

```
fooB:
addi sp sp -8
...
jal x1 fooC
...
addi sp sp 8
jr ra
```

Current Line

Space that **fooC** isn't allowed to change (guaranteed to stay the same)

Address	Data
0xFFFF FF0C	0x12345678
0xFFFF FF08	0x9ABCDEF0
0xFFFF FF04	0x00000000
0xFFFF FF00	0xABADCAFE
0xFFFF FEFC	0x4F639DAB
0xFFFF FEF8	0x14857642

Register	Value
sp	0xFFFF FEFC

Manual Stack Manipulation: Example

```
fooB:
addi sp sp -8
...
jal x1 fooC
...
addi sp sp 8
jr ra
```

Current Line

After **fooC**, sp shouldn't be different

Address	Data
0xFFFF FF0C	0x12345678
0xFFFF FF08	0x9ABCDEF0
0xFFFF FF04	0x00000000
0xFFFF FF00	0xABADCAFE
0xFFFF FEFC	0x4F639DAB
0xFFFF FEF8	0x14857642

Register	Value
sp	0xFFFF FEFC

Manual Stack Manipulation: Example

```
fooB:
addi sp sp -8
...
jal x1 fooC
...
addi sp sp 8
jr ra
```

Current Line

sp needs to be restored to its original value

Address	Data
0xFFFF FF0C	0x12345678
0xFFFF FF08	0x9ABCDEF0
0xFFFF FF04	0x00000000
0xFFFF FF00	0xABADCAFE
0xFFFF FEFC	0x4F639DAB
0xFFFF FEF8	0x14857642

Register	Value
sp	0xFFFF FEFC

Manual Stack Manipulation: Example

```
fooB:
addi sp sp -8
...
jal x1 fooC
...
addi sp sp 8
jr ra
```

Current Line

Data will eventually be overwritten by another stack frame!

Address	Data
0xFFFF FF0C	0x12345678
0xFFFF FF08	0x9ABCDEF0
0xFFFF FF04	0x00000000
0xFFFF FF00	0xDEADBEEF
0xFFFF FEFC	0xC561CCCC
0xFFFF FEF8	0xF00CF00C

Register	Value
sp	0xFFFF FEFC

RISC-V Functions

CHIPXPRT

NCDC

NUST CHIP DESIGN CENTRE

Converting a C function into RISC-V

```
int foo(int i) {
    if(i == 0) return 0;
    int a = i + foo(i-1);
    return a;
}
int j = foo(3);
int k = foo(100);
int m = j+k;
```

- **Step 1:** Define how foo plans to use registers
- Inputs:
  - i: x10
  - We'll call this register "a0“ for "argument"
  - Return Address: x1
  - We'll call this register "ra"
- Output: x10
  - Yes, we'll reuse a0 for the return value
- Stack Pointer: x2
  - Nicknamed "sp"

Converting a C function into RISC-V

```
int foo(int i) {
    if(i == 0) return 0;
    int a = i + foo(i-1);
    return a;
}
int j = foo(3);
int k = foo(100);
int m = j+k;
```

- **Step 1:** Define how foo plans to use registers

- Register that will NOT be changed by foo: **x8, x9**
  - We can still use these registers, as long as they get restored by the end of the function
  - We'll call these registers "s0" and "s1" for "saved“

- Registers that may be changed by function call: **x5**
  - Since foo can change this, anything that calls foo shouldn't save important data in this register
  - We'll call this register "t0" for "temporary"

Converting a C function into RISC-V

```
int foo(int i) {
    if(i == 0) return 0;
    int a = i + foo(i-1);
    return a;
}
int j = foo(3);
int k = foo(100);
int m = j+k;
```

- **Step 1:** Define how foo plans to use registers

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
int foo(int i) {
    …
}
int j = foo(3);
int k = foo(100);
int m = j+k;
```

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
# int foo(int i) {
#     ...
# }

li a0 3                  # int j = foo(3);
jal ra foo               # call foo
mv s0 a0                 # mv rd rs1 sets rd = rs1

# int k = foo(100);
# int m = j+k;
```

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
# int foo(int i) {
#     ...
# }

li a0 3                 # int j = foo(3);
jal ra foo              # call foo
mv s0 a0                # mv rd rs1 sets rd = rs1

li a0 100               # int k = foo(100);
jal ra foo              # call foo
mv s1 a0                # Saves return value in s1

                        # int m = j+k;
```

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
# int foo(int i) {
#     ...
# }

li a0 3                  # int j = foo(3);
jal ra foo               # call foo
mv s0 a0                 # mv rd rs1 sets rd = rs1

li a0 100                # int k = foo(100);
jal ra foo               # call foo
mv s1 a0                 # Saves return value in s1

add a0 s0 s1             # int m = j+k;
```

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
int foo(int i) {
   if(i == 0) return 0;
   int a = i + foo(i-1);
   return a;
}
…
```

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
int foo(int i) {
    if(i == 0) return 0;
    int j = i - 1;
    j = foo(j);
    int a = i + j;
    return a;
}
...
```

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
int foo(int i) {
    if(i == 0) return 0;
    int j = i - 1;
    j = foo(j);
    int a = i + j;
    return a;
}
...
```

Function call will change **ra** and **a0**. Need to save both somewhere

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
foo:                    # int foo(int i)
addi sp sp -4           # Prologue
sw ra 0(sp)             # Prologue
#    if(i == 0) return 0;
#    int j = i - 1;
#    j = foo(j);
#    int a = i + j;
Epilogue:
lw ra 0(sp)             # Epilogue
addi sp sp 4            # Epilogue
#    return a;
...
```

Option 1: Save **ra** on the stack at the start of the function

Then restore **ra** from the stack (and restore the stack) at the end.

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
foo:                    # int foo(int i)
addi sp sp -4
sw ra 0(sp)
mv s0 a0                # Move i
#    if(i == 0) return 0;
#    int j = i - 1;
#    j = foo(j);
#    int a = i + j;
Epilogue:
lw ra 0(sp)
addi sp sp 4
#    return a;
...
```

Option 2: Save a0 in a saved register so it won't get changed by foo's call

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
foo:                    # int foo(int i)
addi sp sp -8
sw ra 0(sp)
sw s0 4(sp)
mv s0 a0                # Move i
#   if(i == 0) return 0;
addi t0 s0 -1           # int j = i - 1;
mv a0 t0
jal ra foo              # j = foo(j);
mv t0 a0
add a0 s0 t0            # int a = i + j;
Epilogue:
lw ra 0(sp)
lw s0 4(sp)
addi sp sp 8
jr ra                   # return a;
...
```

Use **t0** for **j**, and **a0** for **a**. Due to how foo works, we need to move data to/from a0 for function input/output.

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
foo:                    # int foo(int i)
addi sp sp -8
sw ra 0(sp)
sw s0 4(sp)
mv s0 a0                # Move i
#    if(i == 0) return 0;
addi a0 s0 -1           # int j = i - 1;
jal ra foo              # j = foo(j);
mv t0 a0
add a0 s0 a0            # int a = i + j;
Epilogue:
lw ra 0(sp)
lw s0 4(sp)
addi sp sp 8
jr ra                   # return a;
...
```

Alternative: Use **a0** for **j**, and for **a**.
Saves moving from t0 to a0 and back in this particular code.

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
foo:                # int foo(int i)
addi sp sp -8
sw ra 0(sp)
sw s0 4(sp)
mv s0 a0            # Move i
<CODE> # if(i == 0) return 0;
addi a0 s0 -1       # int j = i - 1;
jal ra foo          # j = foo(j);
mv t0 a0
add a0 s0 a0        # int a = i + j;
Epilogue:
lw ra 0(sp)
lw s0 4(sp)
addi sp sp 8
jr ra               # return a;
...
```

Option A: beq s0 x0 Next li a0 0 j Epilogue	Option B: beq s0 x0 Next li a0 0 jr ra
Option C: bne s0 x0 Next li a0 0 j Epilogue	Option D: bne s0 x0 Next li a0 0 jr ra

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
foo:                    # int foo(int i)
addi sp sp -8
sw ra 0(sp)
sw s0 4(sp)
mv s0 a0                # Move i
bne s0 x0 Next          # if i != 0, skip this
li a0 0                 # int a = 0;
j Epilogue              # Go to Epilogue (to restore stack)
Next:
addi a0 s0 -1           # int j = i - 1;
jal ra foo              # j = foo(j);
mv t0 a0
add a0 s0 a0            # int a = i + j;
Epilogue:
lw ra 0(sp)
lw s0 4(sp)
addi sp sp 8
jr ra                   # return a;
...
```

Option A:	Option B:
beq s0 x0 Next li a0 0 j Epilogue	beq s0 x0 Next li a0 0 jr ra
Option C: bne s0 x0 Next li a0 0 j Epilogue	**Option D:** bne s0 x0 Next li a0 0 jr ra

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Converting a C function into RISC-V

```
j main
foo:                                    # int foo(int i)
        addi sp sp -8
        sw ra 0(sp)
        sw s0 4(sp)
        mv s0 a0                        # Move i
        bne s0 x0 Next                  # if i != 0, skip this
        li a0 0                         # int a = 0;
        j Epilogue                      # Go to Epilogue (to restore stack)
        Next:
        addi a0 s0 -1                   # int j = i - 1;
        jal ra foo                      # j = foo(j);
        add a0 s0 a0                    # int a = i + j;
        Epilogue:
        lw ra 0(sp)
        lw s0 4(sp)
        addi sp sp 8
        jr ra                           # return a;
main:
        li a0 3                         # int j = foo(3);
        jal ra foo                      # call foo
        mv s0 a0                        # mv rd rs1 sets rd = rs1
        li a0 100                       # int k = foo(100);
        jal ra foo                      # call foo
        mv s1 a0                        # Saves return value in s1
        add a0 s0 s1                    # int m = j+k;
```

Calling Convention

CHIPXPRT

NCDC

NUST CHIP DESIGN CENTRE

Calling Convention

- When we wrote foo, we chose "roles" for each register based on how we wanted to use them
- In order for someone else to use foo, they would have to know everything in the table on the right
- We could choose to make one of these tables for every function we need to make
- Better solution: Standardize a set of conventions that everyone agrees to follow.

Register	Role in foo
x10 = a0	i, return value
x1 = ra	return address
x2 = sp	stack pointer
x8 = s0	Saved Register
x9 = s1	Saved Register
x5 = t0	Temporary

Calling Convention

- Each register is given a name according to what its role is (no need to memorize the exact mapping):
  - **zero:** The x0 register, which always stores 0
  - **ra:** x1, which is used to store return addresses
  - Two new pseudoinstructions that explicitly use this:
    - jal Label → jal ra Label
    - ret → jr ra

#	Name	Description
x0	zero	Constant 0
x1	ra	Return Address
x2	sp	Stack Pointer
x3	gp	Global Pointer
x4	tp	Thread Pointer
x5	t0	Temporary Registers
x6	t1	
x7	t2	
x8	s0	Saved Registers
x9	s1	
x10	a0	Function Arguments or Return Values
x11	a1	
x12	a2	Function Arguments
x13	a3	
x14	a4	
x15	a5	

#	Name	Description
x16	a6	Arguments
x17	a7	
x18	s2	Saved Registers
x19	s3	
x20	s4	
x21	s5	
x22	s6	
x23	s7	
x24	s8	
x25	s9	
x26	s10	
x27	s11	
x28	t3	Temporaries
x29	t4	
x30	t5	
x31	t6	

Calling Convention

- Callee Saved registers: Registers that must be restored by the end of a function call (i.e. if you want to use it, the called function needs to save the old value)
  - **sp:** The x2 register, which is the stack pointer
  - **s0-s11:** Saved registers

#	Name	Description
x0	zero	Constant 0
x1	ra	Return Address
x2	sp	Stack Pointer
x3	gp	Global Pointer
x4	tp	Thread Pointer
x5	t0	Temporary Registers
x6	t1	
x7	t2	
x8	s0	Saved Registers
x9	s1	
x10	a0	Function Arguments or Return Values
x11	a1	
x12	a2	Function Arguments
x13	a3	
x14	a4	
x15	a5	

#	Name	Description
x16	a6	Arguments
x17	a7	
x18	s2	Saved Registers
x19	s3	
x20	s4	
x21	s5	
x22	s6	
x23	s7	
x24	s8	
x25	s9	
x26	s10	
x27	s11	
x28	t3	Temporaries
x29	t4	
x30	t5	
x31	t6	

Calling Convention

- Caller Saved registers: Registers that do not need to be restored by a called function (i.e. if you want to save a variable in this register, it needs to be saved somewhere before you call another function)
  - **ra**
  - **a0-a7:** Registers used for function arguments
  - **a0**, **a1** also used for function outputs
  - If a function needs more than 8 arguments, can use the stack to store more arguments
  - **t0-t6:** Temporary Registers

#	Name	Description
x0	zero	Constant 0
x1	ra	Return Address
x2	sp	Stack Pointer
x3	gp	Global Pointer
x4	tp	Thread Pointer
x5	t0	Temporary Registers
x6	t1	
x7	t2	
x8	s0	Saved Registers
x9	s1	
x10	a0	Function Arguments or Return Values
x11	a1	
x12	a2	Function Arguments
x13	a3	
x14	a4	
x15	a5	

#	Name	Description
x16	a6	Arguments
x17	a7	
x18	s2	Saved Registers
x19	s3	
x20	s4	
x21	s5	
x22	s6	
x23	s7	
x24	s8	
x25	s9	
x26	s10	
x27	s11	
x28	t3	Temporaries
x29	t4	
x30	t5	
x31	t6	

Calling Convention

- Other registers: Registers that are out of scope for this class (don't use them!)
  - **gp:** The **x3** register, used to store a reference to the heap. Also called the "global pointer"
  - **tp:** The **x4** register, used to store separate stacks for threads)

#	Name	Description
x0	zero	Constant 0
x1	ra	Return Address
x2	sp	Stack Pointer
x3	gp	Global Pointer
x4	tp	Thread Pointer
x5	t0	Temporary Registers
x6	t1	
x7	t2	
x8	s0	Saved Registers
x9	s1	
x10	a0	Function Arguments or Return Values
x11	a1	
x12	a2	Function Arguments
x13	a3	
x14	a4	
x15	a5	

#	Name	Description
x16	a6	Arguments
x17	a7	
x18	s2	Saved Registers
x19	s3	
x20	s4	
x21	s5	
x22	s6	
x23	s7	
x24	s8	
x25	s9	
x26	s10	
x27	s11	
x28	t3	Temporaries
x29	t4	
x30	t5	
x31	t6	

RISC-V Summary

- Over the past four lectures, we've covered almost everything about programming in RISC-V.
- Arithmetic operations allow you to do math with registers
- Immediate versions for register-constant operations
- Loads/Stores for accessing memory
- Branches for conditionally changing the current line of code
- Jumps for function calls and unconditional jumps
- Only a few remaining instructions left!

All remaining RISC-V instructions not covered!

- **slt, slti, sltu, sltiu:** Set Less Than
  - slt rd rs1 rs2: Compares rs1 to rs2. If rs1 < rs2 (signed), sets rd to 1. Otherwise sets rd to 0.

- **ebreak, ecall:** Environment Break/Call
  - Asks the computer to do something (ex. Print data, set a breakpoint for debugging, allocate heap space)
  - We'll provide utility functions that call ecall/ebreak for you

Call / Return in RISC-V

- Calling a procedure in RISC-V
  - **jal x1, ProcedureLabel**
  - Save return address (PC+4) in x1 and jump to ProcedureAddress/Label
  - Used by **caller**!
    - Pseudoinstruction for same is **call ProcedureLabel**
  - **jalr** can also be used for function calls after auipc & addi
- Returning from a procedure
  - **jalr x0, 0(x1)**
  - Jump to address in x1 (return address) and save PC+4 in x0 (but x0 is hardwired to 0 → ignore return address)
  - Used by **callee**!
    - Pseudoinstruction for same is ret
- Unconditional jump without returning
  - Sometimes needed within any procedure
  - **jal x0, Label** # unconditionally branch to Label

Thank You

CHIPXPRT

NCDC
NUST CHIP DESIGN CENTRE